STRUCTURAL BREAKS AND CHANGEPOINT ANALYSIS

GitHub Repository, File Review & Troubleshooting Guide

Purpose: A practical handover guide for project partners. It explains which files matter, how to inspect the repository, which terminal commands to run, how to interpret the outputs, and what to do when a GitHub push fails.

Repository	econometrics-structural-breaks
GitHub remote	https://github.com/Ananyo-Sen-B2530066/econometrics-structural-breaks.git
Default branch	main
Project focus	Monte Carlo simulation, robustness analysis, and Indian CPI application

This guide is written for collaborators who may be new to Git and GitHub. Run commands from the project folder unless a step says otherwise. Read the expected output before deciding what to do next.

1. What the repository contains

The repository stores the code, data, analysis notebook, generated results, figures, and documentation needed to inspect and reproduce the project. The exact filenames in your local checkout take precedence if the folder layout has changed.

Folder / file	What to inspect it for	Why it matters
data/raw/	Original input data, including the CPI workbook (for example, cpi_1133.xlsx).	Shows the source input used in the empirical application. Check provenance and whether redistribution is permitted.
data/processed/	Cleaned analysis-ready data, such as india_cpi_cleaned.csv.	Allows partners to understand the transformed dataset without rerunning every cleaning step.
notebooks/	real_data_application.ipynb and any supporting notebooks.	Contains the empirical workflow, diagnostics, tests, plots, and interpretation.
scripts/	Simulation, robustness, break-testing, and supplementary method scripts.	Contains executable analysis code. Inspect settings and output paths before running.
results/	Raw Monte Carlo and robustness result files.	Stores numerical outputs that support summary tables and figures.
outputs/figures/	Simulation and real-world figures.	Visual evidence for the report and project interpretation.
outputs/tables/	Generated tables, where present.	Provides tabular summaries of results.
README.md	Project overview, methods, findings, setup, limitations.	The first document a visitor or collaborator should read.
requirements.txt	Python dependency list and pinned versions.	Helps collaborators recreate the Python environment.
.gitignore	Patterns Git should not track, such as .venv/ and __pycache__/.	Prevents machine-specific files and bulky environments from entering Git history.

Important project interpretation notes

The empirical CPI application covers All-India, Combined, General CPI from January 2013 to December 2024 (144 monthly observations). The pre-specified April 2020 test is a HAC-robust joint Wald test, not the textbook classical Chow F-test. The reported p-value is approximately 0.0549, which is not significant at the 5% level. The BIC-selected dynamic-programming segmentation selected zero breaks under the tested specifications.

The segmentation procedure is described as Bai–Perron-style dynamic-programming segmentation with BIC; do not describe it as a complete formal implementation of every part of the Bai–Perron framework. HAC inference does not remove serial correlation, and the segmentation objective does not explicitly model it.

2. Before running commands

Open Terminal and move into the project directory. On the original machine, the project folder was located at ~/STRUCTURAL-BREAKS. If your folder is elsewhere, replace the path with your own.

```
cd ~/STRUCTURAL-BREAKS
pwd
```

Check: pwd should print the full path to the project directory. If it prints a different directory, stop and navigate to the correct folder before running Git commands.

Useful first-look commands

```
ls -la
find . -maxdepth 2 -type f | sort
du -sh .
```

Command	What it tells you	What to look for
ls -la	Lists visible and hidden items in the current folder.	README.md, requirements.txt, .gitignore, .git/, and expected project folders.
find . -maxdepth 2 -type f | sort	Lists files up to two folder levels below the project root.	Check that key scripts, notebook, data, and result files are present.
du -sh .	Shows total project folder size.	A surprisingly large size may indicate a virtual environment, cache, or large data file.

3. Files to review before sharing

File	Review checklist
README.md	Does the overview match the actual work? Are methods, sample period, replication counts, findings, limitations, setup commands, and folder names accurate?
requirements.txt	Are all imported third-party packages included? Are versions valid for the Python version used? Avoid listing standard-library modules such as pathlib or time.
.gitignore	Does it exclude .venv/, venv/, __pycache__/, .ipynb_checkpoints/, .DS_Store, local secrets, and temporary files?
Notebook	Does it run top-to-bottom? Are outputs and figures current? Are file paths relative to the repository rather than absolute paths from one laptop?
Scripts	Do imports work? Are random seeds/configurations clear? Are output paths safe? Are expensive replication counts understood before execution?
Data files	Are source, coverage, date range, units, missing-value handling, and permission to share documented?
Results and figures	Do they correspond to the current code and settings? Are filenames understandable? Are plots labelled with units and context?

Do not upload

Do not commit .venv/, passwords, API keys, access tokens, .env files, private data, temporary caches, or unrelated personal files. Large datasets should only be committed if you have permission and the repository's storage approach is appropriate.

4. Git checks: exact commands and how to read them

Run these commands in order. They are read-only checks unless stated otherwise.

A. Confirm the current folder and branch

```
pwd
git branch --show-current
```

Expected: the path ends in STRUCTURAL-BREAKS (or your chosen project directory), and the branch command prints `main`. If the branch is different, do not assume it is wrong; confirm with the project owner before switching.

B. Check repository state

```
git status
```

Output fragment	Meaning / next action
nothing to commit, working tree clean	No uncommitted changes are present. This is a clean state.
Changes not staged for commit	Tracked files were edited but not staged. Review them, then use git add for intended changes.
Changes to be committed	Files are staged and will be included in the next commit. Inspect before committing.
Untracked files	New files are not yet tracked by Git. Decide whether they belong in the project.
Your branch is ahead of 'origin/main' by N commits	Local commits are not yet on GitHub. A push is needed.
Your branch is behind 'origin/main' by N commits	The remote has commits missing locally. Fetch and inspect before integrating.
have diverged	Both local and remote have unique commits. Do not force-push; inspect history and coordinate integration.

C. Inspect the latest local commit

```
git log -1 --oneline
```

This shows the latest commit hash and message. A line such as `c71f388 new addition` means the local branch currently points to commit c71f388. It does not by itself prove that GitHub has received it.

D. Check the remote URL

```
git remote -v
```

Expected remote for this project:

```
origin  https://github.com/Ananyo-Sen-B2530066/econometrics-structural-breaks.git (fetch)
origin  https://github.com/Ananyo-Sen-B2530066/econometrics-structural-breaks.git (push)
```

The fetch and push URLs should point to the intended repository. If the URL is wrong, stop and confirm the correct URL with the repository owner before changing it.

E. Check whether GitHub can be contacted and inspect its branch tip

```
git ls-remote origin
```

This reads references from the remote repository; it does not upload your local commit. A successful result typically contains a hash followed by refs/heads/main. If the command fails with an authentication, DNS, timeout, or server error, record the full message.

5. Compare local and remote history

```
git fetch origin
git status
git log --oneline --decorate --graph --all -10
```

What each command does

Command	Purpose
git fetch origin	Downloads remote branch information without merging it into your working branch.
git status	Shows whether your branch is ahead, behind, diverged, or clean after fetching.
git log --oneline --decorate --graph --all -10	Displays a compact graph of recent commits across local and remote-tracking branches.

Common outcomes

State after fetch	Interpretation	Recommended action
ahead by 1 commit	One local commit is missing from the remote branch.	If the remote is otherwise up to date and the commit is intended, try git push origin main.
up to date	Local main and origin/main point to the same commit.	No push is needed at this moment.
behind by N commits	Remote has new commits not in local main.	Review the incoming commits and coordinate before pulling/rebasing.
diverged	Local and remote have separate commits.	Stop and coordinate. Do not force-push.

6. Push to GitHub

Only push after confirming the branch, remote URL, and local/remote history.

```
git push origin main
```

A successful push commonly ends with output similar to:

```
To https://github.com/Ananyo-Sen-B2530066/econometrics-structural-breaks.git
   <old>..<new>  main -> main
```

This means GitHub accepted the new commit(s). Verify with:

```
git status
git ls-remote origin
```

After a successful push, status should usually say the branch is up to date with origin/main. The hash returned by ls-remote for refs/heads/main should match the local main commit hash shown by git log -1 --oneline.

7. Troubleshooting a rejected push

Error / symptom	What it usually means	Safe next step
remote rejected ... Internal Server Error	A server-side failure or transient GitHub service problem is possible. It does not automatically mean the local commit is lost.	Check GitHub Status, retry once later, and capture the complete output. Do not force-push.
Authentication failed / Permission denied	Credentials, token, SSH key, or repository permission may be incorrect.	Check account access and authentication method. Do not paste tokens or private keys into chat.
Repository not found	The URL may be wrong, or the signed-in account may not have access to a private repository.	Verify the URL and collaborator permissions with the owner.

Error / symptom	What it usually means	Safe next step
fetch first / non-fast-forward	Remote contains commits not present locally.	Fetch and inspect the graph; coordinate before merge or rebase.
file exceeds GitHub limit	A large file may exceed GitHub's standard file-size limit.	Identify the file; consider Git LFS or an external data repository. Do not delete blindly.
RPC failed / connection reset / timeout	Network interruption, proxy, or large transfer may be involved.	Check network and GitHub Status; retry later. Inspect repository size and large files if repeated.

For the specific error in the project handover

The observed state was: local main was ahead of origin/main by one commit; the working tree was clean; local HEAD was c71f388 (message: “new addition”); the remote URL was the intended HTTPS repository; and git ls-remote origin succeeded, returning a main-branch hash. This indicates that the local commit existed and GitHub was reachable for a read operation. The failed push itself was rejected with Internal Server Error. A later retry is reasonable, but the result must be checked rather than assumed.

8. Team workflow: how partners should contribute

Recommended routine for each collaborator

```
cd ~/STRUCTURAL-BREAKS
git switch main
git pull --ff-only origin main
# make your changes

git status
git diff
git add <files-you-intend-to-include>
git diff --cached
git commit -m "Describe your change"
git push origin main
```

This simple workflow assumes one person at a time is updating main and there are no conflicting concurrent changes. For a larger team, use feature branches and pull requests instead of everyone pushing directly to main.

Before staging: inspect the change

```
git status --short
git diff
```

`git status --short` gives a compact list of modified, deleted, and untracked files. `git diff` shows unstaged content changes. After staging, use `git diff --cached` to inspect exactly what will enter the next commit.

Stage only intended files

```
git add README.md requirements.txt
# or stage selected project files by path
git diff --cached --stat
git diff --cached --name-only
```

Avoid `git add .` if you have not checked for private, temporary, or large files. Review the staged filenames and diff before committing.

Commit and publish

```
git commit -m "Update project documentation"
git push origin main
```

A commit saves a snapshot in your local Git history. A push publishes commits to GitHub. A commit does not automatically upload anything.

9. Useful file and size audits

```
# Show tracked files
 git ls-files

# Find files larger than 20 MB (macOS)
find . -type f -not -path './.git/*' -not -path './.venv/*' -size +20M -print

# Confirm virtual environment is ignored
 git check-ignore -v .venv

# Show staged file summary
 git --no-pager diff --cached --stat
```

Interpretation

Command	What to check
git ls-files	Lists files Git currently tracks. Confirm .venv, credentials, and caches are absent.
find ... -size +20M	Finds unusually large files outside .git and .venv. Check whether they are necessary and allowed to be shared.
git check-ignore -v .venv	Should show the .gitignore rule responsible for ignoring the environment. If it prints nothing, inspect .gitignore.

Command	What to check
git diff --cached --stat	Summarizes staged changes by file and number of insertions/deletions.

10. What to send when asking for help

When a command fails, copy the command and the complete output. Do not send only the last line if earlier lines contain the cause.

For a push problem, provide these outputs:

```
git status
git log -1 --oneline
git remote -v
git ls-remote origin
git push origin main
```

For a general repository review, provide:

```
pwd
git branch --show-current
git status --short
git --no-pager diff --stat
git --no-pager diff --cached --stat
git ls-files
```

Privacy and security

Before sharing terminal output, check that it does not contain access tokens, passwords, private keys, personal file paths you do not want to disclose, or private dataset content. Never share the contents of ~/.ssh/id_rsa, ~/.ssh/id_ed25519, credential files, or .env files. It is generally safe to share a GitHub repository URL and ordinary Git status/log output, but review everything first.

11. Quick reference: command → question answered

Question	Command
Am I in the project folder?	pwd
Which branch am I on?	git branch --show-current
Are there uncommitted changes?	git status
What is my latest commit?	git log -1 --oneline
Where does origin point?	git remote -v
Can I read the remote branch refs?	git ls-remote origin
What changed locally?	git diff
What is staged for commit?	git diff --cached
What files does Git track?	git ls-files
Can I publish my local commit?	git push origin main
Did the remote receive my commit?	git status and git ls-remote origin; compare commit hashes

12. Safe-practice checklist

Before every push, confirm:

- ■ Correct project folder and branch.
- ■ Correct remote repository URL.
- ■ No secrets, virtual environments, caches, or unauthorized data staged.
- ■ README and requirements match the actual project.
- ■ Staged changes reviewed with git diff --cached.
- ■ Local/remote history checked if the branch is behind or diverged.
- ■ Commit message clearly describes the change.

■ Push result verified with git status and remote branch hash.

Golden rule: if Git reports a rejected push, preserve the local repository and inspect the state before attempting repairs. Never use git push --force, git reset --hard, or delete .git as a first response to a push error.

Prepared for the Structural Breaks and Changepoint Analysis project team. This is an operational guide; the repository's actual code, data provenance, and current Git state should always be checked before making changes.